Politecnico
di Torino

Ruggine Chat

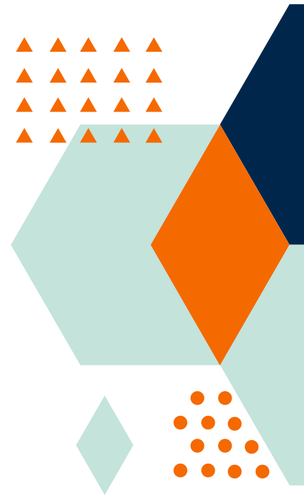
Real-Time Chat Application

Agnese Re s325676
Ilaria Sarcuni s332008
Cosimo Sergi s347914

Programmazione di Sistema

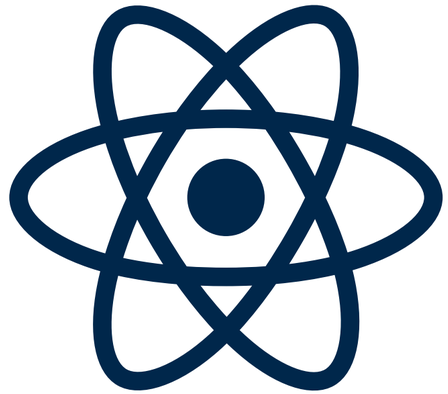
Cos'è Ruggine Chat

- Piattaforma di messaggistica istantanea
- Interazione mediante due differenti modalità:
  - **chat private** per conversazioni dirette uno a uno
  - **chat di gruppo** per conservazioni in team
- Funzionalità
  - registrazione nuovo utente
  - login con username e password
  - creazione chat private e team
  - accettazione e rifiuto inviti team
  - stato online/offline
  - notifiche messaggi



Architettura del sistema

- Architettura **client-server**
  - backend in Rust
    - framework *Axum*
  - frontend in React
- Runtime *tokio*
- Comunicazione real-time via *WebSocket*



Inizializzazione server

- **main.rs**
  - pool di connessioni, politiche di sessione, routes

```
// 6. Routing System
let app: Router = Router::new() Router<AppState>
    .route(path: "/", method_router: get(handler: || async { "Backend Online" })) Router<AppState>
    // Authetication
    .route(path: "/register", method_router: post(handler: h_auth::register)) Router<AppState>
    .route(path: "/login", method_router: post(handler: h_auth::login)) Router<AppState>
    .route(path: "/logout", method_router: get(handler: h_auth::log_out)) Router<AppState>
    .route(path: "/me", method_router: get(handler: h_auth::get_me).route_layer(from_fn(auth_middleware))) Router<AppState>
```

• • •

```
    // Layers
    .layer(AuthSessionLayer::<User, i64, SessionSqlitePool, SqlitePool>::new(pool: Some(pool.clone())).with_config
    (auth_config)) Router<AppState>
    .layer(SessionLayer::new(session_store)) Router<AppState>
    .layer(cors) Router<AppState>
    .with_state(state);
```

Auth Middleware

- Route che richiedono autenticazione
  - insert utente in mappa delle estensioni
    - oppure 401 Unauthorized error

```rust
// Authentication middleware for protected routes
async fn auth_middleware(auth: AuthSession<User, i64, SessionSqlitePool, SqlitePool>,
    mut req: axum::extract::Request, next: axum::middleware::Next) -> impl IntoResponse {

    if auth.is_authenticated() {
        req.extensions_mut().insert(val: auth.current_user.unwrap().clone());
        next.run(req).await
    } else {
        (axum::http::StatusCode::UNAUTHORIZED, "Non autorizzato").into_response()
    }
}
```

All'handler o prossimo miiddleware

Handler example

not authenticated route


```rust
pub async fn register(
    State(state: AppState): State<AppState>,
    Json(user: UserRequest): Json<UserRequest>
) -> Result<impl IntoResponse, AppError> {

    if user.username.trim().is_empty() || user.password.trim().is_empty() {
        return Err(AppError::BadRequest("Username e password richiesti.".into()));
    }

    let exists: bool = sqlx::query_scalar(sql: "SELECT EXISTS(SELECT 1 FROM user WHERE username = ?1)") QueryScalar<'_, Sqlite,
    bool, …>
        .bind(&user.username).fetch_one(executor: &state.pool).await?;

    if exists { return Err(AppError::RegistrationFail); }

    let hash_password: String = bcrypt::hash(user.password, cost: 10).map_err(anyhow::Error::new)?;

    sqlx::query(sql: "INSERT INTO user (username, password) VALUES (?1, ?2)") Query<'_, Sqlite, SqliteArguments<'_>>
        .bind(&user.username).bind(&hash_password).execute(executor: &state.pool).await?;

    Ok((StatusCode::OK, Json(json!({ "success": true, "message": "Registrazione completata!" }))))
}
```

Handler example cont'd

authenticated route

```rust
// --- LIST PRIVATE CHATS (id, other_username, other_user_id) ---
pub async fn get_chat_list(Extension(user: User): Extension<User>,
    State(state: AppState): State<AppState>) -> Result<impl IntoResponse, AppError> {

    let rows: Vec<ChatListRow> = sqlx::query_as(sql: "SELECT c.id,
        CASE
            WHEN c.id_user1 = ?1 THEN u2.username ELSE u1.username
        END as other_username,
        CASE
            WHEN c.id_user1 = ?1 THEN c.id_user2 ELSE c.id_user1
        END as other_user_id
        FROM private_chats_assoc c
        JOIN user u1 ON c.id_user1 = u1.id
        JOIN user u2 ON c.id_user2 = u2.id
        WHERE c.id_user1 = ?1 OR c.id_user2 = ?1") QueryAs<'_, Sqlite, ChatListRow, …>
        .bind(user.id) QueryAs<'_, Sqlite, ChatListRow, …>
        .fetch_all(executor: &state.pool) impl Future<Output = Result<…, …>>
        .await?;

    Ok(Json(rows))
}
```

AppState


```rust
pub type ChatRooms = Arc<DashMap<i64, broadcast::Sender<String>>>;
pub type OnlineUsers = Arc<DashMap<i64, Arc<tokio::sync::Mutex<HashSet<i64>>>>>;
pub type PresenceMap = Arc<DashMap<i64, Instant>>;

#[derive(Clone)]
2 implementations
pub struct AppState {
    pub pool: SqlitePool,
    pub chat_rooms: ChatRooms,
    pub online_users: OnlineUsers,
    pub presence_map: PresenceMap,
}
```

Concorrenza
DashMap, sharding dei lock

Condivisione
Arc, accesso thread-safe

Real-time
Broadcast, push dei messaggi

Database SQLite

Tabella	Attributi	Descrizione
user	id*, username, password	Utenti registrati
team	id*, name	Team creati
user_team	id_user*, id_team*, last_data, last_ora	Associazioni utente-team
invite	id_user*, id_team*, id_invited_by	Inviti pendenti

Database SQLite cont'd

Tabella	Attributi	Descrizione
message	id_message*, id_user, id_team, message, data, ora, type	Messaggi inviati all'interno dei team
private_messages	id_chat, message, data, ora, name1, name2, type	Messaggi inviati in chat private
private_chats_assoc	id*, id_user1, id_user2, last_data_user1, last_ora_user1, last_data_user2, last_ora_user2	Associazioni utente-chat
session_table	id*, expires, session	Sessioni

Routes (auth)

Route	Descrizione
/register	Registrazione di un nuovo utente
/login	Login con creazione di sessione
/logout	Logout con eliminazione sessione
/me	Restituisce dettagli utente

Routes (private chat)

Route	Descrizione
/create/private	Crea una nuova chat privata tra due utenti
/chat/messages/{chat_id}	Restituisce storico messaggi chat privata
/chat/send	Invia un nuovo messaggio all'interno di una chat privata
/mark-read-private/{chat_id}	Contrassegna come letti i messaggi di una chat privata

Routes (team)

Route	Descrizione
/create	Crea un nuovo team
/team/{team_id}/members	Restituisce l'elenco completo dei membri di un team
/send	Invia un nuovo messaggio all'interno di un team
/mark-read/{team_id}	Contrassegna come letti i messaggi di un team

WebSocket

- Connessioni bidirezionali e persistenti tra client e server
  - in tempo reale
    - senza polling
- HTTP upgrade
- 3 endpoint
  - /ws/team/{id}
  - /ws/private/{id}
  - /ws/global

WebSocket cont'd


```rust
async fn handle_socket(socket: WebSocket, state: AppState, room_id: i64) {

    let tx: Sender<String> = state.chat_rooms Arc<DashMap<i64, Sender<Str…
        .entry(key: room_id) Entry<'_, i64, Sender<String>>
        .or_insert_with(|| broadcast::channel(capacity: 100).0) RefMut<'_, i64, Sender<String>>
        .clone();
    let mut rx: Receiver<String> = tx.subscribe();
    let (mut sender: SplitSink<WebSocket, Message>, mut receiver: SplitStrea…) = socket.split();

    let mut send_task: JoinHandle<()> = tokio::spawn(future: async move {
        while let Ok(msg: String) = rx.recv().await {
            if sender.send(item: Message::Text(msg.into())).await.is_err() {
                break;
            }
        }
    });

    let mut recv_task: JoinHandle<()> = tokio::spawn(future: async move {
        while let Some(Ok(msg: Message)) = receiver.next().await {
            if let Message::Close(_) = msg {
                break;
            }
        }
    });

    tokio::select! {
        _ = &mut send_task => recv_task.abort(),
        _ = &mut recv_task => send_task.abort(),
    }

} async fn handle_socket
```

Monitoraggio risorse

- Task asincrono gestito tramite runtime *tokio*
- Percentuale utilizzo CPU e tempo di attività processo
  - ogni 120 secondi
  - crate *sysinfo*

```rust
if let Some(process: &Process) = sys.process(pid) {
    let usage: f32 = process.cpu_usage();
    let run_time: u64 = process.run_time();
    let num_cpus: usize = sys.cpus().len();

    let timestamp: String = Local::now().format(fmt: "%d-%m-%Y %H:%M:%S").to_string();
    let log_entry: String = format!(
        "[{}] CPU Usage: {:.2}% | Runtime: {}s | Cores: {}\n",
        timestamp, usage, run_time, num_cpus
    );
```

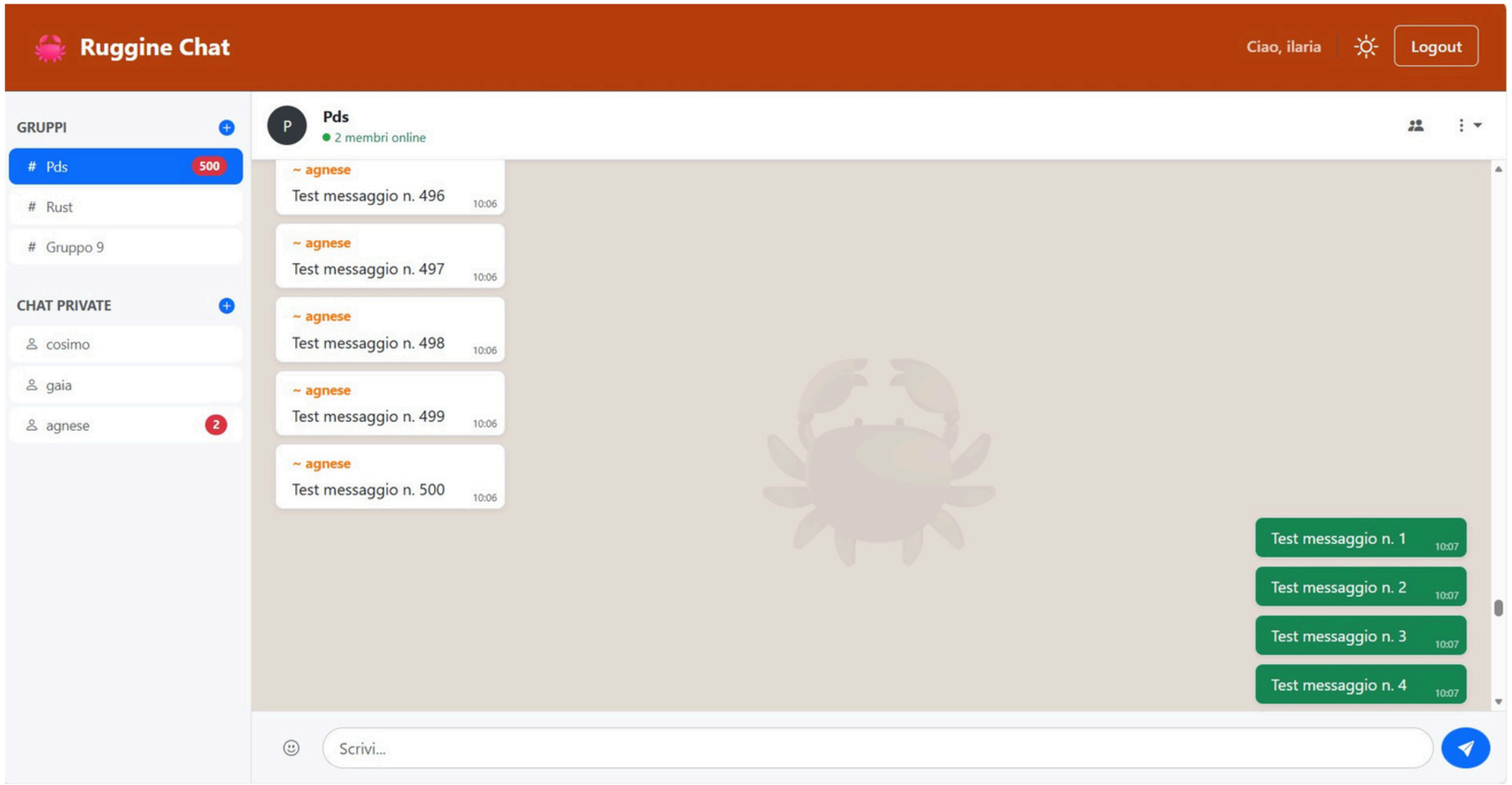
Su backend/cpu_log.txt

Monitoraggio risorse cont'd

- In condizioni idle: ~ 0.01% - 0.50%
- In sessioni di test: ~ **0.50% - 3.00%**
- In stress test: ~ 5.00%
  - script JS automatizzati in *DevTools*

```
async function stressTest(numeroMessaggi, delayMs) {
    const teamId = 1;
    const endpoint = 'http://localhost:3000/send';
    // repeat numeroMessaggi
    const response = await fetch(endpoint, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        credentials: 'include',
        body: JSON.stringify({team_id: teamId, message: `Test messaggio n. ${i}`})
    });
```

Monitoraggio risorse cont'd



Conclusioni

- Il backend risulta estremamente leggero anche sotto carico.
- Il merito è di tre scelte architetturali:
  - Rust
    - nessun garbage collector
    - memoria gestita a compile time
  - Tokio asincrono
    - un thread gestisce migliaia di connessioni WS senza bloccarsi
  - DashMap
    - HashMap concorrente senza lock globali sullo stato condiviso
- Dimensione binario release backend.exe: **2.27 MB**

Grazie

per l’attenzione